

Formal Methods for Cyber-Physical Systems 2024/2025

Project

March 4, 2025

Assignment	Max Grade	Achieved Grade
Report (PDF max 10 pages)	8	
Exercise 1	12	
Exercise 2	12	
Final grade	32	

1 Verification of Reactivity Properties

In the first part of the project you will implement a symbolic algorithm for the verification of a special class of LTL formulas, using BDDs as data structure to represent and manipulate regions. The class of formulas you will considered is the one of *reactivity properties* that have the special form $\Box\Diamond f \rightarrow \Box\Diamond g$, where f and g are Boolean combination of base formulas with no temporal operators. Some examples of reactivity properties for the Railroad Controller are listed below.

- If the west train is repeatedly waiting, then it will be repeatedly on the bridge:

$$\Box\Diamond train_w.mode = wait \rightarrow \Box\Diamond train_w.mode = bridge$$

- If the west train is repeatedly waiting then repeatedly either the corresponding signal is green or the east train is on the bridge:

$$\Box\Diamond train_w.mode = wait \rightarrow \Box\Diamond (signal_w = green \vee train_e.mode = bridge)$$

- If the west train is repeatedly waiting with the east train off the bridge, then the west train will be repeatedly on the bridge:

$$\Box\Diamond (\neg (train_e.mode = bridge) \wedge train_w.mode = wait) \rightarrow \Box\Diamond train_w.mode = bridge$$

- The following formulas are not reactivity properties, and should be discarded:

$$\begin{aligned} & \Box (train_w.mode = wait \rightarrow \Diamond signal_w = green) \\ & \Box\Diamond \neg (train_e.mode = bridge) \rightarrow \Box\Diamond train_w.mode = wait \rightarrow \Box\Diamond train_w.mode = bridge \\ & \Box\Diamond train_w.mode = wait \rightarrow \Box\Diamond (signal_w = green \wedge Otrain_w.mode = bridge) \end{aligned}$$

1.1 Assignment

The archive `react_mc.zip` contains the schema of the code and some SMV programs to test your solution. The file `react_mc.smv` is a Python script that uses the `pynusmv` library to verify reactivity properties of SMV programs. The script uses the function `parse_react(spec)` to check if `spec` is a reactivity formula, and skips all formulas that are not in the correct form. Using the `react_mc.smv` script as a starting point, re-implement the function `check_react_spec(spec)`, respecting the following specifications:

- the function checks if the reactivity formula `spec` is satisfied by the loaded SMV model or not, that is, whether all the executions of the model satisfy `spec` or not;
- the return value of `check_react_spec(spec)` is a tuple where the first element is `True` and the second element is `None` if the SMV model satisfies the formula. When the formula is not satisfied, the first element is `False` and the second element is an execution of the SMV model that violates `spec`. The function returns `None` if `spec` is not a reactivity formula;
- the execution is a tuple of alternating states and inputs, starting and ending with a state. States and inputs are represented by dictionaries where keys are state and inputs variable of the loaded SMV model, and values are their value;
- the execution is looping: the last state should be somewhere else in the sequence to indicate the starting point of the loop.

1.2 The pynusmv library

`pynusmv` is a Python wrapper to NuSMV model checking algorithms. The library consists of several modules. To implement the project you can use only the following modules: `init`, `glob`, `fsm` (all methods except `reachable_states`), `prop`, `dd`. You can also use the helper function `spec_to_bdd(model, spec)` included in `react_mc.py` to convert a boolean property with no temporal operators to an equivalent BDD, and the helper function `parse_react(spec)` that checks if `spec` is a reactivity property. If `spec` is a reactivity formula $\Box\Diamond f \rightarrow \Box\Diamond g$, `parse_react(spec)` returns a pair where the first element is the formula f and the second element is the formula g . If `spec` is not a reactivity formula, the function returns `None`.

You can find more information about the `pynusmv` library at the websites <http://lvl.info.ucl.ac.be/Tools/PyNuSMV> and <https://pynusmv.readthedocs.io>. Binary files to install `pynusmv` on recent Python versions are available at <https://github.com/davidebreso/pynusmv/releases/latest>.

2 Supervisory control

Consider an elevator that can move across 3 storeys. The plant of the elevator consists of the following modules:

- A cabin that can go up and down according to the floor it is on. In its initial state the cabin is on the ground floor.
- A door that can be either open or closed. In its initial state the door is closed and thus it can only be opened. Once the door is open it can only be closed.
- A button at every floor to “call” the elevator. Once a floor is called, pushing the button again does not change the fact that the floor has been called.

2.1 Assignments

- Define a finite state automaton for each component above also justifying your choices regarding the controllability of events and the marking of the states.
- Define a finite state automaton for each of these requirements justifying your choices regarding the controllability of events and the marking of the states.

R_1) The cabin can move only if the door is closed.

R_2) If a floor has been called and the cabin is on that floor, then the elevator must serve that floor. Serving a floor means that the elevator must open and close the door.

- Implement this exercise with ESCET. Specify the plant and the requirements with CIF. Design a simple SVG graphical interface that reacts to the execution of the events. Use ToolDef to synthesise the supervisor enforcing R_1 and R_2 and deploy your supervisor. Check that the behaviour of the controlled plant is the expected one.

3 How to submit the project

Your submission should include:

- The complete Python code of part 1
- The complete CIF code, SVG and ToolDef of part 2
- A short report describing the details and implementation choices you made. For the first part, the report should describe how your code generate the counterexample, and should justify the correctness of the algorithm. For the second part, the report should contain the automata formalising the plant and the requirements, and a discussion justifying the modelling choices.

Upload your submission on Moodle, using the “Sumbission of the alternative project” activity in section “Alternative Project”.